

2. Változók és Adattípusok C#-ban

Mi az a változó?

A változó egy névvel ellátott tárterület a memóriában, ahol adatokat tárolhatunk. Minden változónak van **típusa**, amely meghatározza, milyen adatot tárolhat, és mennyi memóriát foglal.

Alapvető adattípusok (Value Types)

Egész számok (Integer típusok)

```
// byte - 8 bit, előjel nélküli (0 - 255)
byte kor = 25;
byte minimumByte = 0;
byte maximumByte = 255;

// sbyte - 8 bit, előjeles (-128 - 127)
sbyte homerseklet = -15;
sbyte minSbyte = -128;
sbyte maxSbyte = 127;

// short - 16 bit, előjeles (-32,768 - 32,767)
short populacio = 15000;
short minShort = -32768;
short maxShort = 32767;

// ushort - 16 bit, előjel nélküli (0 - 65,535)
ushort portSzam = 8080;

// int - 32 bit, előjeles (-2,147,483,648 - 2,147,483,647)
int lakasAr = 50000000;
int minInt = -2147483648;
int maxInt = 2147483647;

// uint - 32 bit, előjel nélküli (0 - 4,294,967,295)
uint nagyMennyiseg = 3000000000;

// long - 64 bit, előjeles
long vilagPopulacio = 8000000000L; // Figyeld az 'L' utótagot!
long minLong = -9223372036854775808L;
long maxLong = 9223372036854775807L;

// ulong - 64 bit, előjel nélküli
ulong nagyonNagySzam = 18000000000000000000UL;
```

Gyakorlati tipp: A legtöbb esetben elég az `int` típus. Csak akkor használj mást, ha:

- Memóriát akarsz spórolni (byte, short)

- Nagyon nagy számokra van szükség (long)
- Biztos, hogy nem lesz negatív érték (unsigned típusok)

Lebegőpontos számok (Floating-point típusok)

```
// float - 32 bit, ~7 számjegy pontosság
float piKorulbelul = 3.14f; // Figyeld az 'f' utótagot!
float ar = 99.99f;
float minFloat = -3.402823e38f;
float maxFloat = 3.402823e38f;

// double - 64 bit, ~15-16 számjegy pontosság (alapértelmezett lebegőpontos típus)
double pontosPi = 3.14159265358979;
double tudomanyosSzam = 1.23e-4; // 0.000123
double minDouble = -1.7976931348623157e308;
double maxDouble = 1.7976931348623157e308;

// decimal - 128 bit, ~28-29 számjegy pontosság (pénzügyi számításokhoz!)
decimal szamlaOsszeg = 1234.56m; // Figyeld az 'm' utótagot!
decimal pontosPenzOsszeg = 99.999999999999999999999999m;
```

Fontos különbség:

```
float f1 = 0.1f + 0.2f;           // 0.30000001 (pontatlan!)
double d1 = 0.1 + 0.2;           // 0.30000000000000004 (pontatlan!)
decimal dec1 = 0.1m + 0.2m;      // 0.3 (pontos!)

// Ezért pénzügyi számításoknál MINDIG decimal-t használj!
decimal egyEuroArfolyam = 390.50m;
decimal osszeg = 100m * egyEuroArfolyam; // 39050.00
```

Logikai típus (Boolean)

```
// bool - true vagy false
bool igazE = true;
bool hamisE = false;

bool vanJogositvanyom = true;
bool nagykoruVagyok = true;
bool vezethetek = vanJogositvanyom && nagykoruVagyok; // true

// Használat feltételekben
if (vezethetek)
{
    Console.WriteLine("Vezethetsz!");
}
```

Karakter típus (Character)

```
// char - 16 bit, egyetlen Unicode karakter
char betu = 'A';
char szam = '5';
char specKarakter = '@';
char ujSor = '\n';
char tab = '\t';

// Unicode karakterek
char sziv = '♥';
char emoji = '😄'; // Vigyázat: egyes emoji-k 2 char-t foglalnak!

// Karakter kódja
char a = (char)65; // 'A'
int kod = (int)'A'; // 65
```

Referencia típusok

String (karakterlánc)

```
// string - szöveges adatok tárolására, hossza változó
string nev = "Kovács János";
string uresString = "";
string nullString = null; // Nincs értéke

// String műveletek
string keresztnev = "Anna";
string vezeteknev = "Nagy";
string teljesNev = vezeteknev + " " + keresztnev; // "Nagy Anna"

// String interpoláció (C# 6.0+)
int életkor = 30;
string uzenet = $"A nevem {nev}, {életkor} éves vagyok.";
Console.WriteLine(uzenet); // A nevem Kovács János, 30 éves vagyok.

// String hossza
int hossz = nev.Length; // 12

// String immutable (megváltozhatatlan)
string s1 = "Hello";
string s2 = s1.ToUpper(); // "HELLO"
// s1 továbbra is "Hello" marad!
```

Változók deklarálása és inicializálása

1. Deklarálás értékadás nélkül

```
int szam; // Deklarálva, de nincs inicializálva
// Console.WriteLine(szam); // HIBA! Használat előtt értéket kell adni!

szam = 10; // Most már használható
Console.WriteLine(szam); // 10
```

2. Deklarálás inicializálással

```
int szam = 42;
string nev = "Péter";
bool aktiv = true;
```

3. Típus következtetés (var kulcsszó)

```
var szam = 10;           // int típus lesz
var nev = "Anna";       // string típus lesz
var ar = 99.99;          // double típus lesz
var ar2 = 99.99m;        // decimal típus lesz
var igaz = true;         // bool típus lesz

// A var csak akkor használható, ha azonnal inicializálsd!
// var x; // HIBA!
```

Mikor használjuk a var-t?

```
// Egyszerű esetekben olvashatóbb az explicit típus
int kor = 25; // Jobban olvasható
var kor2 = 25; // Kevésbé egyértelmű

// Bonyolult típusoknál a var hasznos
var lista = new List<Dictionary<string, int>>(); // Sokkal egyszerűbb!
List<Dictionary<string, int>> lista2 = new List<Dictionary<string, int>>(); //
Hosszú
```

4. Több változó egyszerre

```
// Ugyanolyan típusúak
int a = 1, b = 2, c = 3;

// Vagy csak deklarálás
int x, y, z;
x = 10;
```

```
y = 20;  
z = 30;
```

5. Alapértelmezett értékek

```
int szam = default;           // 0  
bool igaz = default;         // false  
string szoveg = default;     // null  
double d = default;          // 0.0  
  
// Új C# verziókban (7.1+)  
int szam2 = default(int);    // 0
```

Típuskonverzió

1. Implicit konverzió (Automatikus)

Akkor történik, ha **nincs adatvesztés** (kisebb típusból nagyobbba).

```
// Egész számok  
byte b = 100;  
int i = b;           // byte → int (OK)  
long l = i;          // int → long (OK)  
  
// Lebegőpontos  
float f = 3.14f;  
double d = f;        // float → double (OK)  
  
// Egészből lebegőpontosba  
int szam = 100;  
double lebego = szam; // int → double (OK)  
Console.WriteLine(lebego); // 100.0
```

2. Explicit konverzió (Casting)

Amikor **lehet adatvesztés**, explicit konverzióra van szükség.

```
// Nagyobb típusból kisebbbe  
double d = 123.456;  
int i = (int)d;           // 123 (a tizedesjegyek elvesznek!)  
  
long nagy = 300;  
byte kicsi = (byte)nagy;  // 300 → 44 (túlcsordulás!)  
  
// Float → int  
float f = 9.99f;
```

```
int egesz = (int)f;           // 9 (nem kerekít, levágja!)

// Veszélyes példák
int nagyInt = 300;
byte b = (byte)nagyInt;      // 44 - adatvesztés, túlcsordulás!

double d2 = 1.9999;
int i2 = (int)d2;           // 1 (nem kerekít!)
```

3. Convert osztály használata

Biztonságosabb és olvashatóbb konverziók.

```
// String → szám
string szamSzoveg = "123";
int szam = Convert.ToInt32(szamSzoveg);           // 123
double lebege = Convert.ToDouble("45.67");        // 45.67
decimal penz = Convert.ToDecimal("99.99");        // 99.99
bool ertek = Convert.ToBoolean("true");            // true

// Szám → string
int kor = 25;
string korSzoveg = Convert.ToString(kor);         // "25"

// Kerekítéssel konvertál
double d = 1.9;
int kerekitve = Convert.ToInt32(d);               // 2 (kerekít!)
int castolva = (int)d;                            // 1 (levág!)

// Hibakezelés
try
{
    int szam = Convert.ToInt32("nem szám"); // Kivételt dob!
}
catch (FormatException)
{
    Console.WriteLine("Hibás formátum!");
}
```

4. Parse és TryParse metódusok

String-ből típusra konvertálás.

```
// Parse - kivételt dob hiba esetén
string szoveg1 = "42";
int szam1 = int.Parse(szoveg1);                 // 42
double szam2 = double.Parse("3.14");            // 3.14
bool igaz = bool.Parse("true");                 // true
```

```
// Hiba esetén kivétel
try
{
    int hiba = int.Parse("abc"); // FormatException!
}
catch (FormatException)
{
    Console.WriteLine("Nem lehet konvertálni!");
}

// TryParse - biztonságosabb, nem dob kivételt
string szoveg2 = "123";
int eredmeny;
bool sikeres = int.TryParse(szoveg2, out eredmeny);

if (sikeres)
{
    Console.WriteLine($"Konverzió sikeres: {eredmeny}"); // 123
}
else
{
    Console.WriteLine("Konverzió sikertelen!");
}

// Még rövidebb (C# 7.0+)
if (int.TryParse("456", out int szam3))
{
    Console.WriteLine($"Sikerült: {szam3}");
}

// Rossz input kezelése
string rosszInput = "abc";
if (int.TryParse(rosszInput, out int szam4))
{
    Console.WriteLine($"Érték: {szam4}");
}
else
{
    Console.WriteLine("Hibás bemenet!"); // Ez fog lefutni
}
```

Melyiket használjam?

- **TryParse**: Ha nem vagy biztos, hogy valid a bemenet (pl. felhasználói input)
- **Parse**: Ha biztos vagy benne, hogy valid az input
- **Convert**: Univerzális konverzióhoz, több típus között

Konstansok

Olyan értékek, amelyek **nem változtathatók meg** a program futása során.

1. const kulcsszó

Fordítási időben ismert értékek.

```
// Alapvető konstansok
const int MaxJatekosok = 10;
const double Pi = 3.14159265359;
const string Valuta = "HUF";
const bool DebugMod = true;

// Használat
int jatekosokSzama = 8;
if (jatekosokSzama <= MaxJatekosok)
{
    Console.WriteLine("Lehet még csatlakozni!");
}

// Hiba: nem módosítható!
// MaxJatekosok = 20; // FORDÍTÁSI HIBA!

// Konstansok osztályban
class Beallitasok
{
    public const int MinJelszohossz = 8;
    public const int MaxBejelentkezes = 3;
    public const string AppNev = "MyCoolApp";
}

// Használat
if (jelszo.Length < Beallitasok.MinJelszohossz)
{
    Console.WriteLine("A jelszó túl rövid!");
}
```

2. readonly kulcsszó

Futási időben ismert értékek, de csak konstruktorban állíthatók be.

```
class Konfiguracio
{
    // const - fordítási időben
    public const string AppVerzio = "1.0.0";

    // readonly - futási időben, de csak egyszer állítható be
    public readonly string Utvonal;
    public readonly DateTime InditasiIdo;

    public Konfiguracio(string utvonal)
    {
        Utvonal = utvonal; // OK, konstruktorban
```

```

        InditasiIdo = DateTime.Now; // OK, futási idő alatti érték
    }

    public void Modositas()
    {
        // Utvonal = "új útvonal"; // HIBA! Nem módosítható!
    }
}

// Használat
var config = new Konfiguracio("C:\\App");
Console.WriteLine(config.Utvonal); // "C:\\App"
// config.Utvonal = "D:\\App"; // HIBA!

```

const vs readonly összehasonlítás

```

class Pelda
{
    // const
    public const int Konstans = 100;
    // - Fordítási időben ismert
    // - static implicit módon
    // - Csak primitív típusok és string

    // readonly
    public readonly int SzamlaloKezdo;
    public readonly DateTime Letrehozva;

    public Pelda(int kezdoErtek)
    {
        SzamlaloKezdo = kezdoErtek;           // OK
        Letrehozva = DateTime.Now;           // OK - futási időben kap értéket
    }

    // readonly lehet objektum is
    public readonly List<string> Nevek = new List<string>();

    public void NevHozzaadas(string nev)
    {
        // Nevek = new List<string>(); // HIBA! Nem lehet új lista
        Nevek.Add(nev); // OK! A lista tartalma módosítható
    }
}

```

Gyakorlati összefoglaló példa

```

class Program
{

```

```
// Osztály szintű konstansok
const double AfaKulcs = 0.27;
const string Valuta = "Ft";

static void Main(string[] args)
{
    // Adattípusok használata
    string termékNev = "Laptop";
    int mennyiseg = 2;
    decimal nettoAr = 150000m;

    // Számítások
    decimal afaOsszeg = nettoAr * (decimal)AfaKulcs;
    decimal bruttoAr = nettoAr + afaOsszeg;
    decimal vegosszeg = bruttoAr * mennyiseg;

    // Kimenet
    Console.WriteLine($"Termék: {termékNev}");
    Console.WriteLine($"Mennyiség: {mennyiseg} db");
    Console.WriteLine($"Nettó ár: {nettoAr:N0} {Valuta}");
    Console.WriteLine($"ÁFA ({AfaKulcs * 100}%): {afaOsszeg:N0} {Valuta}");
    Console.WriteLine($"Bruttó ár: {bruttoAr:N0} {Valuta}");
    Console.WriteLine($"Végösszeg: {vegosszeg:N0} {Valuta}");

    // Típuskonverzió példa
    Console.WriteLine("\nAdd meg a kedvezményt (%): ");
    string input = Console.ReadLine();

    if (double.TryParse(input, out double kedvezmeny))
    {
        decimal kedvezmenyOsszeg = vegosszeg * (decimal)(kedvezmeny / 100);
        decimal fizetendo = vegosszeg - kedvezmenyOsszeg;

        Console.WriteLine($"Kedvezmény: {kedvezmenyOsszeg:N0} {Valuta}");
        Console.WriteLine($"Fizetendő: {fizetendo:N0} {Valuta}");
    }
    else
    {
        Console.WriteLine("Hibás kedvezmény érték!");
    }
}
```

Összefoglalás

- **Value types:** int, double, bool, char - közvetlen érték tárolása
- **Reference types:** string - memóriacím tárolása
- **Deklarálás:** Típus + név + opcionális inicializálás
- **var:** Típus következtetés, csak inicializálással
- **Implicit konverzió:** Automatikus, ha nincs adatvesztés
- **Explicit konverzió (casting):** Manuális, adatvesztés lehet

- **Convert:** Biztonságos konverzió több típus között
 - **Parse/TryParse:** String → típus konverzió
 - **const:** Fordítási időben ismert konstans
 - **readonly:** Futási időben beállítható, de később nem módosítható
-

C# Alapok Tananyag - 2. Fejezet